AKADEMIA NAUK STOSOWANYCH
W NOWYM SĄCZU

Wydział Nauk Inżynieryjnych
Katedra Informatyki

DOKUMENTACJA PROJEKTOWA

ZAAWANSOWANE PROGRAMOWANIE

Algorytm drzewa BST
z zastosowaniem GitHub

Autor:
Jakub Pyrdoł
Tymoteusz Zaczyk
Dawid Plewa

Prowadzący:
mgr inż. Dawid Kotlarski

Nowy Sącz 2026

Spis treści

1. Ogólne określenie wymagań **3**
1.1. Przykład . . . 4
1.2. Instalacja . . . 5
1.3. Wykorzystanie Git i GitHub . . . 6
2. Analiza problemu **10**
2.1. Zastosowanie i reprezentacja danych . . . 10
2.2. Sposób działania algorytmu . . . 10
2.3. Ręczny przykład działania . . . 11
2.4. Zależności między modułami a kolejność scalania . . . 12
3. Projektowanie **13**
3.1. Podział odpowiedzialności . . . 13
3.2. Warunki poprawności drzewa . . . 13
3.3. Menu konsolowe . . . 14
3.4. Dane zapisywane w plikach . . . 15
4. Implementacja **16**
4.1. Dodawanie i usuwanie węzłów . . . 16
4.2. Wyszukiwanie, przejścia i rysunek . . . 17
4.3. Sprawdzanie danych podczas odczytu . . . 17
4.4. Sprawdzenie działania . . . 18
5. Wnioski **19**
Literatura **20**
Spis rysunków **21**
Spis tabel **22**
Spis listingów **23**

1. Ogólne określenie wymagań

Celem projektu jest przygotowanie binarnego drzewa poszukiwań (BST, ang. *Binary Search Tree*) przechowującego liczby całkowite. Program ma przedstawiać sposób organizowania danych w strukturze drzewiastej oraz umożliwiać dodawanie i usuwanie elementów z zachowaniem określonego porządku. Projekt obejmuje również wykorzystanie systemu kontroli wersji Git oraz platformy GitHub do przechowywania kodu i zapisywania kolejnych etapów jego rozwoju.

Drzewo składa się z węzłów połączonych relacjami rodzic–dziecko. Węzeł rozpoczynający strukturę nazywany jest korzeniem. Każdy węzeł drzewa binarnego może mieć najwyżej dwoje dzieci: lewe i prawe. Węzeł bez dzieci jest liściem, natomiast węzły znajdujące się poniżej danego elementu tworzą jego poddrzewa. Taki sposób organizacji danych pozwala opisywać zależności pomiędzy elementami bez ograniczania ich do jednej kolejności liniowej.

W drzewie BST obowiązuje dodatkowa reguła porządku. Wszystkie wartości lewego poddrzewa są mniejsze od wartości rozpatrywanego węzła, a wszystkie wartości prawego poddrzewa są większe. Ta sama zasada obowiązuje na każdym poziomie drzewa. Dzięki temu porównanie wartości pozwala wybrać dalszy kierunek przejścia: w lewo albo w prawo. W bieżącym projekcie jednakowe wartości nie są przechowywane wielokrotnie.

Drzewa poszukiwań są przydatne przy organizowaniu zbiorów kluczy oraz danych, dla których istotne są porównania i relacja porządku. Praca nad taką strukturą pozwala poznać zależność pomiędzy przyjętym sposobem przechowywania danych a działaniem operacji wykonywanych na tych danych. W tym projekcie istotne jest zarówno poprawne utworzenie połączeń, jak i ich zachowanie po dodaniu lub usunięciu elementu.

Węzły są przechowywane w kontenerze `std::vector<Node>`. Każdy z nich zawiera liczbę całkowitą oraz indeksy lewego i prawego dziecka. Połączenia nie są więc zapisane za pomocą wskaźników do oddzielnych obiektów. Pozycja elementu w wektorze określa miejsce przechowywania, natomiast zapisane indeksy określają jego miejsce w logicznej strukturze drzewa.

Dokumentacja opisuje implementację projektu Drzewo-BST, którego kod jest utrzymywany w repozytorium na GitHubie [1]. Program umożliwia wykonywanie następujących operacji.

- utworzenie pustego drzewa, do którego można następnie dodawać kolejne wartości,
- dodawanie elementów we właściwym miejscu drzewa z pomijaniem wartości już występujących,
- usuwanie węzła o wskazanej wartości z aktualizacją połączeń pomiędzy pozostałymi elementami,
- opróżnianie całego drzewa i ponowne wykorzystanie tego samego obiektu,
- wyświetlanie rysunku drzewa i wartości w kolejności preorder, inorder albo postorder,
- wyszukiwanie ścieżki od korzenia do podanej wartości,
- zapis do pliku tekstowego lub binarnego, odtworzenie drzewa z pliku binarnego i dodawanie liczb z pliku tekstowego.

Program rozpoczyna pracę z pustym drzewem. Funkcja `main()` wyświetla menu konsolowe, a użytkownik wybiera operację przez podanie jej numeru. Tabelę opcji i zasady wprowadzania danych podano w podrozdziale 3.3.

Sposób organizacji pracy z użyciem Git i GitHuba opisano w podrozdziale 1.3.

1.1. Przykład

Po uruchomieniu programu należy wybrać opcję `1` i wprowadzić kolejno wartości 1, 2, 3, 8 oraz 5. Znak `q` kończy dodawanie i przywraca menu. Opcja `5` wypisuje wartości w porządku rosnącym oraz wyświetla rysunek drzewa.

Pierwsza wartość staje się korzeniem. Kolejne liczby są porównywane z wartościami już znajdującymi się w drzewie i kierowane do odpowiedniego poddrzewa. Liczby 2, 3 i 8 tworzą w tym przykładzie ciąg prawych połączeń. Wartość 5 zostaje natomiast umieszczona jako lewe dziecko węzła 8. Dobór danych pozwala więc przedstawić oba kierunki połączeń.

Przejście inorder wypisuje $1, 2, 3, 5, 8$. Po wybraniu opcji `2`, podaniu wartości `3` i powrocie przez `q` to samo przejście daje $1, 2, 5, 8$. Zmiany połączeń i indeksów

podczas usuwania opisano w podrozdziale 2.3. Opcja `3` opróżnia całe drzewo. Można wtedy dodać nową wartość, na przykład 42, w tej samej sesji.

1.2. Instalacja

Do kompilacji wymagany jest kompilator obsługujący standard C++14. Program korzysta z biblioteki standardowej C++, w szczególności z kontenera `vector` oraz strumieni `iostream`. Nie wymaga instalowania dodatkowych bibliotek. Za przygotowanie budowania odpowiada plik `CMakeLists.txt`, który określa nazwę celu `Drzewo_BST` i wymaga CMake w wersji co najmniej 4.1.

Do pobrania kodu z GitHuba należy użyć klienta Git. Poniższe polecenia tworzą lokalną kopię repozytorium i przechodzą do katalogu projektu.

```
git clone https://github.com/pyr-jak/Drzewo-BST.git
cd Drzewo-BST
```

W katalogu projektu należy przygotować konfigurację budowania.

```
cmake -S . -B build
```

Opcja `-S .` wskazuje bieżący katalog jako miejsce przechowywania źródeł, natomiast `-B build` określa katalog, w którym CMake umieści pliki związane z budowaniem aplikacji. Pozwala to oddzielić źródła od wyników kompilacji.

Następnie należy zbudować program.

```
cmake --build build
```

Polecenie korzysta z wcześniej przygotowanej konfiguracji i uruchamia właściwe narzędzie budowania. Powstały plik `Drzewo_BST`, a w systemie Windows zwykle `Drzewo_BST.exe`, znajduje się w katalogu budowania lub jego podkatalogu zależnym od generatora i konfiguracji.

W systemach Linux i macOS, przy generatorze z jednym wariantem budowania, program można uruchomić poleceniem `./build/Drzewo_BST`. Polecenie uruchomione z katalogu źródeł powoduje, że program odczytuje i zapisuje pliki o stałych nazwach w tym katalogu. W trakcie pracy z drzewem dane wprowadza się przez menu.

1.3. Wykorzystanie Git i GitHub

Git zapisuje kolejne stany projektu jako commity. Gałąź wskazuje ostatni commit danej linii rozwoju. CLion udostępnia operacje Git w interfejsie środowiska, a GitHub przechowuje zdalne repozytorium projektu [1]. Pull request (PR) umożliwia zespołowi przegląd i zatwierdzenie zmian przed ich włączeniem do gałęzi głównej [2].

1.3.1. Gałąź główna i trzy gałęzie robocze

Tabela 1.1 przedstawia przykładowy podział pracy nad projektem. Łącznie obejmuje on cztery gałęzie. Wspólna gałąź `main` zawiera wersję zintegrowaną, a trzy gałęzie robocze służą do rozwijania odrębnych części programu. Każdą z nich tworzy się z aktualnej wersji `main`.

Tab. 1.1. Gałąź główna i trzy gałęzie robocze

Gałąź	**Zakres prac**
`main`	Gałąź główna, do której trafiają zatwierdzone zmiany.
`pyr-branch`	Branch jednego z twórców.
`zak-branch`	Branch jednego z twórców.
`ple-branch`	Branch jednego z twórców.

1.3.2. Tworzenie i publikowanie gałęzi w CLionie

Jeżeli repozytorium jest już sklonowane, należy otworzyć jego katalog jako projekt. Kopię można też utworzyć w CLionie, wybierając na ekranie powitalnym *Clone Repository*, wklejając adres repozytorium GitHub i wskazując katalog docelowy.[1]

W nowym repozytorium punktem wyjścia jest pierwszy commit na gałęzi `main`. Jeśli lokalna gałąź początkowa ma inną nazwę, można użyć polecenia *Rename* w widżecie gałęzi i opublikować `main` w skonfigurowanym repozytorium `origin`.

1. Kliknąć nazwę bieżącej gałęzi na pasku CLiona, aby otworzyć widżet kontroli wersji. Wybrać lokalną gałąź `main` i polecenie *Checkout*. Jeżeli dostępna jest tylko gałąź zdalna `origin/main`, wybrać ją i użyć *Checkout*, aby rozpocząć pracę na lokalnej gałęzi.

2. Na gałęzi `main` użyć polecenia *Git* → *Pull* i pobrać zmiany z `origin/main`.

3. W widżecie gałęzi wybrać *New Branch*, wpisać `pyr-branch` i zaznaczyć *Checkout branch*, aby od razu przełączyć się na nową gałąź. Przy tworzeniu `zak-branch` i `ple-branch` za każdym razem najpierw wrócić na `main` i sprawdzić jej aktualność.[2]

4. Wprowadzić zmiany na właściwej gałęzi. W oknie *Commit* przejrzeć różnice, zaznaczyć pliki należące do zadania i zatwierdzić je z opisowym komunikatem. Następnie wybrać *Git* → *Push*. W oknie wysyłania sprawdzić, czy celem jest gałąź o tej samej nazwie w repozytorium `origin`. Pierwsze wysłanie tworzy gałąź na GitHubie.[3]

Przed przełączeniem się na kolejną gałąź należy zakończyć pracę z bieżącymi zmianami, na przykład zatwierdzając je na właściwej gałęzi. Nazwa widoczna w widżecie pozwala sprawdzić, do której linii rozwoju trafi kolejny commit.

[1] JetBrains, Set up a Git repository.
[2] JetBrains, Manage Git branches.
[3] JetBrains, Commit and push changes to Git repository.

1.3.3. Scalanie zmian na stronie GitHub

Dla każdej opublikowanej gałęzi roboczej otwiera się osobny PR do `main`. Procedura obejmuje następujące kroki.

1. W repozytorium na GitHubie otworzyć zakładkę *Pull requests* i wybrać *New pull request*. W polu *base* ustawić `main`, a w polu *compare* wybrać gałąź roboczą.
2. Sprawdzić listę różnic i wybrać *Create pull request*. Podać zakres zmian i wyniki prób. Osoba recenzująca sprawdza pliki w zakładce *Files changed*. Dodatkowe commity wysłane z CLiona na tę samą gałąź aktualizują otwarty PR [2].
3. Po akceptacji i spełnieniu wymaganych kontroli wybrać *Create a merge commit*, jeżeli repozytorium udostępnia tę metodę. Następnie użyć *Merge pull request* i *Confirm merge*. Commit scalający zachowuje historię poszczególnych commitów oraz zaznacza punkt połączenia gałęzi [3].

Jeśli gałęzie wprowadzają zależne zmiany, w pierwszej kolejności scala się operacje BST, następnie obsługę plików, a na końcu menu wywołujące te operacje. Przed scaleniem kolejnego PR-a należy uwzględnić w jego gałęzi bieżący stan `main` i sprawdzić kompilację oraz działanie powiązanych funkcji. Zależności między modułami omówiono w podrozdziale 2.4. Po scaleniu wszystkich PR-ów należy przełączyć CLiona na `main` i pobrać wynik przez *Git* → *Pull*.

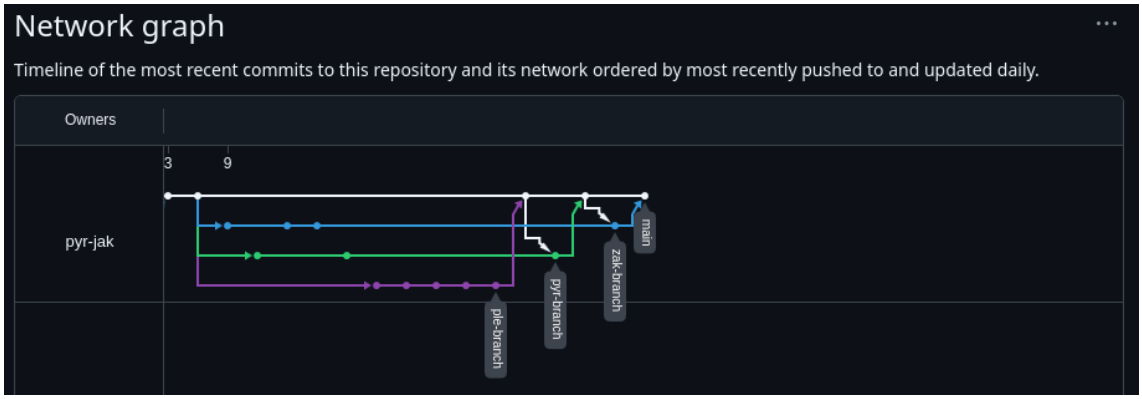
1.3.4. Aktualizacja gałęzi i rozwiązywanie konfliktów

Jeżeli GitHub pokazuje przycisk *Update branch*, można go użyć, aby włączyć aktualny stan `main` do gałęzi PR-a. Tę aktualizację można również wykonać w CLionie. Należy przełączyć się na gałąź roboczą, wybrać *Git* → *Fetch*, a następnie w widżecie gałęzi wskazać `origin/main` i wybrać operację *Merge* wskazującą bieżącą gałąź roboczą jako cel.

Jeżeli Git wykryje konflikt, CLion pozwala porównać obie wersje i uzgodnić wynik w edytorze scalania. Na przykład przy konflikcie w `CMakeLists.txt` należy sprawdzić, czy ostateczna lista plików zawiera wszystkie jednostki kompilacji potrzebne do zbudowania programu. Po zastosowaniu rozstrzygnięcia trzeba ukończyć scalanie, przeprowadzić próby i wysłać wynik na gałąź roboczą przez *Push*. GitHub zaktualizuje PR, który można następnie scalić do `main` na stronie repozytorium [4].

1.3.5. Historia zmian w widoku Network Graph

Po opublikowaniu gałęzi i scaleniu pull requestów można udokumentować przebieg prac na wykresie historii repozytorium. Na stronie projektu należy przejść do zakładki *Insights*, a następnie wybrać *Network*. Na zrzucie warto uwzględnić przebieg gałęzi `main`, odgałęzienia i punkty scalenia widoczne dla tego zakresu historii.[4]



Rys. 1.1. Historia gałęzi i punktów scalenia w widoku GitHub Network

[4] GitHub Docs, Understanding connections between repositories.

2. Analiza problemu

2.1. Zastosowanie i reprezentacja danych

Drzewo BST służy do porządkowania danych według klucza. Porównanie z wartością bieżącego węzła pozwala wybrać jedno z poddrzew, zamiast sprawdzać wszystkie elementy. Struktura ta może być wykorzystywana do przechowywania zbiorów unikatowych wartości i budowy prostych indeksów.

W projekcie węzeł `Node` zawiera wartość `value` oraz pola `left` i `right`. Pola dzieci są indeksami w wektorze `tree`; wartość `-1` oznacza brak dziecka. Indeks korzenia znajduje się w `rootIndex`. Dla pustego drzewa również wynosi on `-1`.

Rozmieszczenie węzłów w wektorze jest oddzielone od ich połączeń. Element może leżeć pod dowolnym poprawnym indeksem, o ile pola dzieci prowadzą do właściwych poddrzew. Sama kolejność wektora nie musi być posortowana.

2.2. Sposób działania algorytmu

Dodawanie rozpoczyna się od korzenia. Wartość mniejsza jest kierowana do lewego poddrzewa, a większa do prawego. Po dojściu do pustego połączenia nowy węzeł zostaje dopisany na końcu wektora, a jego indeks zapisany jako połączenie od rodzica. Jeżeli wartość już występuje, operacja nie dodaje duplikatu.

Usuwanie rozpoczyna się od wyszukania wartości i zapamiętania rodzica znalezionego węzła. Gdy wartości nie ma, drzewo pozostaje bez zmian. Dla znalezionego elementu rozpatrywane są trzy przypadki:

- liść zostaje odłączony od rodzica;
- węzeł z jednym dzieckiem zostaje zastąpiony połączeniem do tego dziecka;
- węzeł z dwojgiem dzieci otrzymuje wartość minimum prawego poddrzewa, po czym usuwany jest węzeł tego następnika.

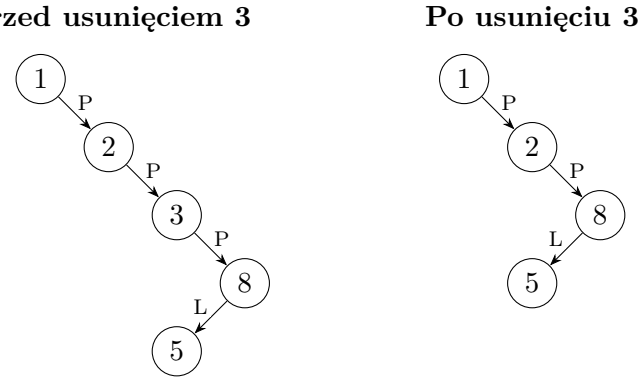
Usunięcie pozycji wektora przesuwa późniejsze elementy. Program zmniejsza więc o jeden wszystkie indeksy większe od usuniętej pozycji, uwzględniając również indeks korzenia. Metoda `removeTree()` czyści cały wektor i ustawia `rootIndex` na `-1`. Obiekt może następnie przyjmować kolejne wartości.

Dla wysokości drzewa h przejście po jednej ścieżce ma koszt $O(h)$. Dodawanie, wyszukiwanie wartości i wyszukiwanie ścieżki zaczynają się od korzenia i korzystają

z tej reguły. Drzewo nie jest równoważone, więc przy niekorzystnej kolejności danych wysokość może rosnąć liniowo z liczbą węzłów n. Skuteczne usunięcie ma koszt $O(n)$ ze względu na przesuwanie elementów i poprawianie indeksów. Realokacja bufora wektora może również nadać pojedynczemu dodaniu koszt $O(n)$.

2.3. Ręczny przykład działania

Po dodaniu wartości $1, 2, 3, 8, 5$ korzeniem jest 1. Węzły 2, 3 i 8 tworzą ciąg prawych połączeń, natomiast 5 jest lewym dzieckiem węzła 8. Rysunek 2.1 pokazuje tę strukturę oraz wynik usunięcia wartości 3. Litery L i P oznaczają lewe i prawe dziecko.



Rys. 2.1. Drzewo przed usunięciem wartości 3 i po jej usunięciu

Węzeł o wartości 3 ma jedno dziecko: węzeł 8. Program łączy więc rodzica, czyli węzeł 2, bezpośrednio z węzłem 8. Następnie usuwa pozycję o indeksie 2 z wektora. Węzły 8 i 5 przesuwają się pod indeksy 2 i 3, dlatego trzeba poprawić prowadzące do nich połączenia.

Tabela 2.1 przedstawia zawartość wektora przed operacją i po korekcie indeksów. Korzeń nadal ma indeks 0. Kreski w ostatnim wierszu oznaczają pozycję, której po usunięciu nie ma już w kontenerze.

Tab. 2.1. Zmiana zawartości wektora po usunięciu wartości 3

	Przed usunięciem			**Po usunięciu**		
Indeks	**value**	**left**	**right**	**value**	**left**	**right**
0	1	−1	1	1	−1	1
1	2	−1	2	2	−1	2
2	3	−1	3	8	3	−1
3	8	4	−1	5	−1	−1
4	5	−1	−1	—	—	—

Po tej operacji metoda `show()` wypisałaby $1, 2, 8, 5$, zgodnie z kolejnością w wektorze. Opcja inorder w menu podąża za połączeniami i wypisuje wartości w kolejności $1, 2, 5, 8$.

2.4. Zależności między modułami a kolejność scalania

Jeżeli obie gałęzie robocze wprowadzają zależne zmiany, najpierw należy scalić do `main` operacje drzewa. Na przykład zapis binarny używa metody `preorderValues()`, a menu korzysta z `findPath()`. Zmiany w deklaracjach tych metod powinny trafić do gałęzi z interfejsem, zanim jej PR zostanie scalony. W tym celu włącza się do niej aktualny stan `main` zgodnie z procedurą z punktu 1.3.4.

Po aktualizacji gałęzi interfejsu sprawdza się kompilację programu i uruchamia próby zależnych operacji. Dla `findPath()` można porównać ścieżkę do znanego węzła, a dla zapisu binarnego zapisać drzewo, opróżnić je i odtworzyć z pliku. Wyniki tych prób można dołączyć do PR-a jako podstawę przeglądu.

3. Projektowanie

3.1. Podział odpowiedzialności

Program składa się z interfejsu konsolowego, klasy reprezentującej drzewo oraz modułu operacji plikowych. Tabela 3.1 pokazuje, za co odpowiada każda część. Deklaracje struktur i metod umieszczono w plikach nagłówkowych. Ich szczegółowy wykaz zawiera również dokumentacja wygenerowana przez Doxygen [5].

Tab. 3.1. Podział programu na moduły

Pliki	**Odpowiedzialność**
`main.cpp`	Wyświetla menu, sprawdza dane wejściowe i przekazuje operacje do `BST` lub `BSTFile`.
`BST.h`, `BST.cpp`	Definiują węzeł `Node` i klasę `BST`. Zarządzają węzłami, wyszukiwaniem, przejściami, rysunkiem drzewa i jego zapisem tekstowym.
`BSTFile.h`, `BSTFile.cpp`	Zawierają metody statyczne do zapisu i odczytu binarnego oraz do importu tekstowego. Korzystają z metod publicznych klasy `BST`.
`CMakeLists.txt`	Definiuje budowanie programu `Drzewo_BST` z trzech plików `.cpp`.

3.2. Warunki poprawności drzewa

Wektor `tree` zawiera wszystkie węzły należące do drzewa. W drzewie niepustym `rootIndex` wskazuje prawidłową pozycję w wektorze. Każdy pozostały węzeł jest osiągalny z korzenia przez jedno połączenie od rodzica. Pola dzieci przyjmują wartość -1 albo indeks istniejącego węzła. Przez całą ścieżkę obowiązuje porządek BST opisany w rozdziale 2.

Indeks jest szczegółem reprezentacji i może zmienić się po usunięciu pozycji z wektora. Z tego powodu metody publiczne przyjmują klucze typu `int`, a `findPath()` i `preorderValues()` zwracają wartości węzłów. Program główny może więc używać tych metod bez dostępu do wektora i indeksów.

3.3. Menu konsolowe

Opcje menu zestawiono w tabeli 3.2. W trybach dodawania i usuwania można podać wiele wartości rozdzielonych białymi znakami. Wpisanie małej litery `q` przywraca menu. Wyszukiwanie ścieżki przyjmuje jedną wartość i po wyświetleniu wyniku wraca do menu.

Tab. 3.2. Opcje menu programu

Opcja	**Działanie**	**Metoda**
1	Dodawanie elementów	`BST::addNode()`
2	Usuwanie elementów	`BST::removeNode()`
3	Usuwanie całego drzewa	`BST::removeTree()`
4, 5, 6	Przejścia preorder, inorder i postorder. Każda opcja wypisuje również rysunek drzewa.	`preorder()`, `inorder()`, `postorder()`, `printTree()`
7	Wyszukanie ścieżki od korzenia	`BST::findPath()`
8	Zapis do `drzewo.txt`	`BST::saveToTextFile()`
9	Zapis do `drzewo.bin`	`BSTFile::saveBinary()`
10	Odtworzenie drzewa z `drzewo.bin`	`BSTFile::loadBinary()`
11	Dodanie liczb ze wskazanego pliku tekstowego	`BSTFile::loadText()`
12	Zakończenie programu	Powrót z `main()`

Numery opcji i wartości są odczytywane jako pojedyncze tokeny. Program sprawdza, czy cały token można odczytać jako liczbę typu `int`. Wpisy takie jak `3x`, `3.5` i liczby spoza zakresu typu są odrzucane. Liczba całkowita spoza zakresu opcji 1–12 powoduje ponowne wyświetlenie menu z komunikatem o błędnej opcji. Zakończenie strumienia wejściowego kończy pracę programu.

3.4. Dane zapisywane w plikach

Zapis tekstowy tworzy plik `drzewo.txt` z jedną liczbą w każdym wierszu. Zapis binarny tworzy plik `drzewo.bin`, który zaczyna się od licznika typu `std::uint64_t`, po którym występują kolejne wartości typu `int`. W obu przypadkach wartości są pobierane w kolejności preorder. Plik binarny zapisany przez program zawiera więc $8 + n \cdot$ `sizeof(int)` bajtów dla n węzłów. Program zapisuje typy w ich reprezentacji maszynowej. Przy wymianie plików między platformami wymagane są zgodne rozmiary i kolejność bajtów zapisywanych typów.

Wstawianie wartości w kolejności preorder do pustego drzewa odtwarza jego kształt. Pierwsza wartość staje się korzeniem. Przed wartościami każdego poddrzewa występuje jego własny korzeń, więc ta sama zasada działa na kolejnych poziomach. Węzły w wektorze mogą po odczycie mieć inne indeksy, ponieważ wynikowy wektor jest budowany w kolejności wstawiania.

Odczyt binarny zastępuje bieżące drzewo wynikiem po sprawdzeniu całego pliku. Import tekstowy dopisuje unikatowe liczby do bieżącego drzewa. W opcji 11 należy podać ścieżkę w jednym wierszu. Ścieżka może zawierać spacje i jest podawana bez cudzysłowów. Ścieżki względne są rozwiązywane względem katalogu roboczego programu. Jeśli celem jest odtworzenie drzewa z pliku `drzewo.txt`, należy najpierw je opróżnić opcją 3.

4. Implementacja

4.1. Dodawanie i usuwanie węzłów

Metoda `addNode()` zapamiętuje rozmiar wektora i wywołuje rekurencyjną metodę `insert()`. Wzrost rozmiaru oznacza dodanie nowego węzła i wynik `true`. Wartość już obecna w drzewie kończy operację wynikiem `false`. Po powrocie z wywołania rekurencyjnego rodzic jest ponownie odczytywany przez indeks. Dzięki temu dodanie elementu pozostaje poprawne także po przeniesieniu bufora wektora. Pokazuje to listing 1.

```
25      if (value < tree[currentIndex].value) {
26          int nextLeft = insert(tree[currentIndex].left, value);
27          tree[currentIndex].left = nextLeft;
28      } else if (value > tree[currentIndex].value) {
29          int nextRight = insert(tree[currentIndex].right, value);
30          tree[currentIndex].right = nextRight;
31      }
32
33      return currentIndex;
34  }
```

Listing 1. Aktualizacja połączenia po wstawieniu do poddrzewa

Metoda `removeNode()` wyszukuje iteracyjnie wartość i zapamiętuje indeks rodzica. W przypadku dwojga dzieci szuka następnika, schodząc najpierw w prawo, a następnie maksymalnie w lewo. Kopiuje jego wartość do usuwanego węzła, a fizycznie usuwa następnika. Następnik może mieć prawe dziecko, które przejmuje jego dotychczasowe miejsce. W przypadku usuwania korzenia z najwyżej jednym dzieckiem aktualizowane jest bezpośrednio pole `rootIndex`.

Metoda usuwa pozycję z wektora dopiero po przełączeniu połączeń. Potem zmniejsza wszystkie indeksy większe od usuniętej pozycji. Zwraca `true` po usunięciu węzła i `false`, jeśli klucza nie znaleziono. Metoda `removeTree()` wywołuje `tree.clear()` i ustawia `rootIndex` na -1. Wektor zachowuje pojemność do ponownego wykorzystania.

4.2. Wyszukiwanie, przejścia i rysunek

Metoda `findElement()` zwraca informację o obecności klucza. Używana przez menu metoda `findPath()` dodatkowo zapisuje wartości napotkanych węzłów. Znaleziona ścieżka obejmuje korzeń i szukany węzeł. Gdy wyszukiwanie dojdzie do indeksu -1, metoda zwraca pusty wektor. Jeżeli szukanym kluczem jest korzeń, wynik zawiera jedną wartość.

`preorderValues()` używa wektora indeksów jako stosu. Pobiera węzeł, dopisuje jego wartość do wyniku, a następnie odkłada na stos najpierw prawe, potem lewe dziecko. Dzięki temu lewe poddrzewo jest odwiedzane wcześniej. Z wyniku tej metody korzystają przejście preorder i oba tryby zapisu. Inorder odwiedza lewe poddrzewo, węzeł i prawe poddrzewo, uzyskując wartości rosnąco. Postorder wypisuje węzeł po obu poddrzewach. Te dwa przejścia są rekurencyjne. Metoda `show()` jest osobną funkcją klasy i odczytuje wartości w fizycznej kolejności wektora.

`printTree()` wywołuje rekurencyjną metodę `drawBranch()`, która tworzy etykiety z wartości liczbowych, przydziela im miejsce w kolejności inorder i buduje wiersze rysunku. Między poziomami drzewa umieszcza wiersze z łącznikami. Dla węzła z dwojgiem dzieci pozycja środka etykiety jest wyznaczana na podstawie pozycji obu dzieci. Dla pustego drzewa `printTree()` wypisuje komunikat `Drzewo jest puste.`

4.3. Sprawdzanie danych podczas odczytu

`loadBinary()` zaczyna od odczytania licznika, a następnie wstawia kolejne wartości do nowego obiektu `BST`. Wykrywa brak wymaganych danych, duplikaty i nadmiarowe bajty. Dopiero po sprawdzeniu końca pliku przenosi wynik do drzewa używanego w menu. Poprawny zapis pustego drzewa zawiera sam licznik o wartości zero. Wczytanie takiego pliku opróżnia bieżące drzewo.

`loadText()` zaczyna od kopii istniejącego drzewa. Czyta tokeny oddzielone białymi znakami i sprawdza, czy każdy mieści się w zakresie `int` i składa się w całości z zapisu liczby całkowitej. Duplikaty są pomijane przez `addNode()`. Pusty plik tekstowy zostaje odczytany poprawnie i nie dodaje żadnych wartości. Przy błędzie odczytu albo błędnym tokenie bieżące drzewo zachowuje swój stan także wtedy, gdy wcześniej przetworzono już część pliku. Metody zapisu zwracają błąd przy problemie z otwarciem, zapisem lub zamknięciem pliku. Oba zapisy otwierają plik w trybie nadpisywania, więc błąd po otwarciu może pozostawić plik częściowy.

4.4. Sprawdzenie działania

Kod skompilowano w trybie C++14. Do pustego drzewa wprowadzono kolejno liczby 8, 3, 10, 1, 6, 14, 4, 7 i 13. Listing 2 zawiera wiersze z wynikami trzech przejść odczytane z konsoli.

```
Preorder: 8 3 1 6 4 7 10 14 13
Inorder: 1 3 4 6 7 8 10 13 14
Postorder: 1 4 7 6 3 13 14 10 8
```

Listing 2. Wyniki trzech przejść drzewa

Tabela 4.1 zestawia pozostałe próby wykonane z menu. Próby importu wykonano z drzewa złożonego z węzła o wartości 8.

Tab. 4.1. Próby wykonane w menu

Próba	**Zaobserwowany wynik**
Ponowne dodanie liczby 6	Program zgłosił duplikat.
Wyszukanie 7 i 42	Ścieżka do 7 to $8 \to 3 \to 6 \to 7$. Program zgłosił brak 42.
Usunięcie korzenia 8	Następnikiem został 10. Przejście inorder dało $1, 3, 4, 6, 7, 10, 13, 14$.
Zapis tekstowy i binarny	Plik tekstowy zawierał dziewięć wartości w kolejności preorder. Odczyt zapisu binarnego po opróżnieniu drzewa przywrócił ten sam wynik preorder i ten sam rysunek.
Odczyt obciętego pliku binarnego	Program zgłosił błąd, a drzewo zachowało dotychczasową zawartość.
Import tekstu `-4 8 9`	Przejście inorder dało $-4, 8, 9$. Powtórzona wartość 8 wystąpiła raz.
Import tekstu `-4 9x`	Program zgłosił błąd. Drzewo nadal zawierało tylko wartość 8.

5. Wnioski

Indeksowa reprezentacja drzewa pozwoliła przechowywać węzły w jednym wektorze i oddzielić ich rozmieszczenie od logicznych połączeń. Konsekwencją tego wyboru jest aktualizacja indeksów po usunięciu pozycji. Nawet gdy węzeł można znaleźć szybko, udane usunięcie wymaga przejścia po pozostałych elementach wektora i ma koszt $O(n)$.

Wyszukiwanie wartości i ścieżki oraz porównania podczas wstawiania zależą od wysokości drzewa h. Kolejność danych ma więc znaczenie. Jeżeli liczby są podawane rosnąco, każdy następny węzeł staje się prawym dzieckiem poprzedniego. Wówczas h rośnie liniowo, a budowanie drzewa przez kolejne wstawienia może wymagać $O(n^2)$ czasu. Dotyczy to również odtwarzania takiego drzewa z pliku binarnego. Rekurencyjne wstawianie, inorder, postorder i rysowanie korzystają ze stosu wywołań o głębokości rzędu h.

Każde z trzech przejść odwiedza wszystkie węzły i ma koszt czasowy $O(n)$, bez uwzględnienia rysunku konsolowego. Implementacja preorder gromadzi cały wynik w wektorze i wykorzystuje na ten cel $O(n)$ dodatkowej pamięci.

Próby z podrozdziału 4.4 pokazują, że kolejność przejścia można dobrać do celu. Inorder daje uporządkowaną listę kluczy, a preorder wystarcza do zapisania i odtworzenia kształtu drzewa bez przechowywania indeksów. W obu trybach odczytu wynik trafia do bieżącego drzewa po sprawdzeniu danych. Dzięki temu błędny plik nie pozostawia częściowo wczytanego drzewa.

Naturalnym rozszerzeniem przy wymianie danych między różnymi platformami byłby format binarny z określoną kolejnością bajtów i stałą szerokością klucza. Przy pracy z dużymi, uporządkowanymi zbiorami danych istotnym kierunkiem rozwoju byłoby ograniczenie wysokości drzewa i głębokości rekurencji.

Literatura

[1] Jakub Pyrdoł. Repozytorium projektu Drzewo-BST. https://github.com/pyr-jak/Drzewo-BST.

[2] GitHub Docs. GitHub flow. https://docs.github.com/en/get-started/using-github/github-flow.

[3] GitHub Docs. Pull request merges. https://docs.github.com/en/pull-requests/reference/pull-request-merges.

[4] GitHub Docs. Merge conflicts. https://docs.github.com/en/pull-requests/reference/merge-conflicts.

[5] Programowanie zaawansowane BST. Dokumentacja interfejsu wygenerowana przez Doxygen, plik `doxygen_BST.pdf`.

Spis rysunków

1.1. Historia gałęzi i punktów scalenia w widoku GitHub Network 9
2.1. Drzewo przed usunięciem wartości 3 i po jej usunięciu 11

Spis tabel

1.1. Gałąź główna i trzy gałęzie robocze 6
2.1. Zmiana zawartości wektora po usunięciu wartości 3 12
3.1. Podział programu na moduły 13
3.2. Opcje menu programu 14
4.1. Próby wykonane w menu 18

Spis listingów

1. Aktualizacja połączenia po wstawieniu do poddrzewa 16
2. Wyniki trzech przejść drzewa 18